AI for Autonomous Robots

Practical Assignment: 2D Checkpoint Exploration using Deep Reinforcement Learning

AI4R Practical Assignment

Learning Aim

The aim of this practical assignment is to understand how deep reinforcement learning (DRL) can be used to train an autonomous mobile robot to explore a custom simulated environment. Students will train a PPO agent by default to navigate a continuous 2D multi-room map, cross checkpoint gates, avoid walls, and explain the learned behavior using both quantitative training logs and visual rollouts.

Learning Objectives

After completing this practical, students should be able to:

- understand how a custom Gymnasium-style environment exposes actions, observations, rewards, and episode termination to a DRL algorithm;
- train a PPO-based agent using Ray RLlib and inspect the resulting training logs;
- explain how PPO hyperparameters and training choices affect exploration behavior;
- compare the required entropy or exploration settings and describe how exploration changes;
- evaluate a trained policy using a visual rollout and explain both successful behavior and failure cases.

Assignment Type and Expected Time

Type	Practical exercise
Expected time investment	8–10 hours
Main method	PPO-based deep reinforcement learning
Environment	`2d_checkpoint_exploration` task

Task Overview

The robot moves in a continuous 2D map containing rooms, walls, doorways, and checkpoint gates. Walls are shown in black. Unvisited checkpoint gates are shown in red. Once a checkpoint has been crossed, it turns green. Yellow rays show the robot's range observations. The rays stop at the nearest visible wall or checkpoint intersection and should not pass through walls.

The map contains 20 checkpoint gates. Each new checkpoint gives 0.5 reward:

$$20 \text{ checkpoints} \times 0.5 = 10.0$$

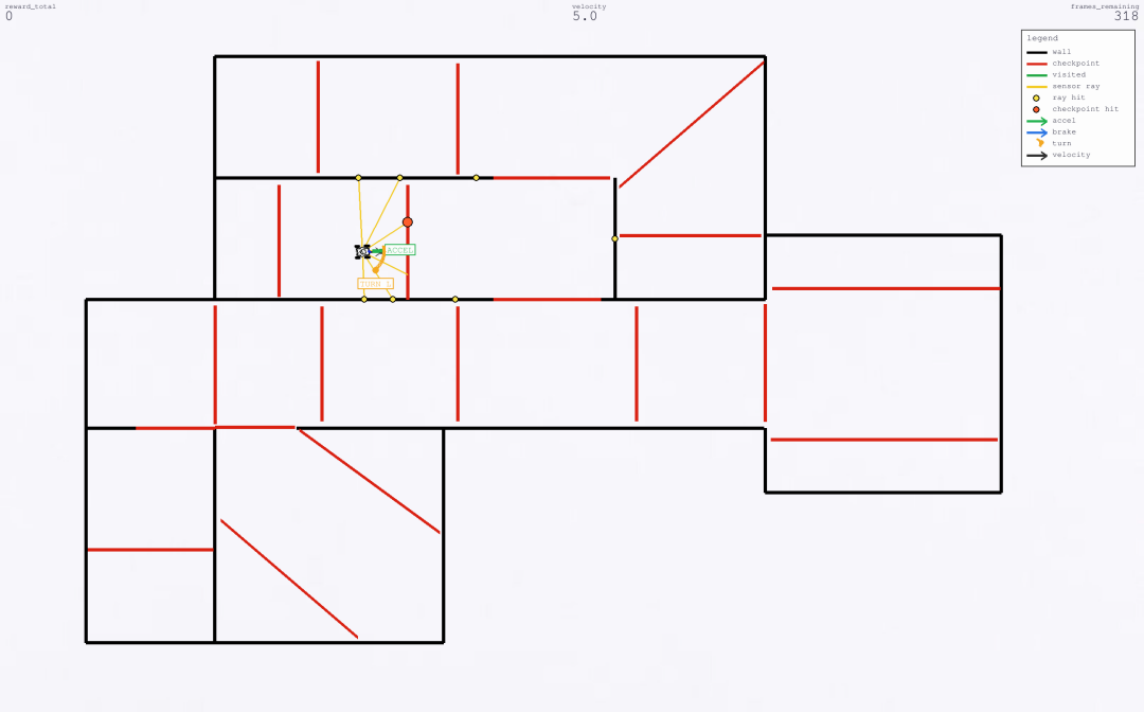

Figure 1: Pygame snapshot of the current 2D checkpoint exploration task. The robot, range rays, checkpoints, walls, and legend are rendered by the simulator.

The goal is to reach as high a score as possible. A score above 6.0 at any point during the allowed 1000-step rollout is enough for submission, as long as the behavior is explained. The final reward may be lower if the robot later collides, stalls, or accumulates penalties. Report both the best score reached during the rollout and the final score if it is different.

The episode ends early if the robot crosses all 20 checkpoint gates. This prevents the policy from losing extra reward by wandering after the exploration task is complete.

Training samples from a small set of fixed start poses by default. This reduces memorization of one start point while keeping the task map fixed. Rollouts use a fixed start by default so videos are easier to compare; use `-spawn-mode random` during rollout to inspect behavior from the training start distribution.

Use the provided PPO command as a baseline, then research and tune `entropy_coeff` and other PPO hyperparameters to reach more than 6.0 during a 1000-step rollout. After selecting the best setup, run one comparison with `entropy_coeff=0.0` and explain the behavior difference between the no-entropy version and the best setup.

Important Rules

- The wall layout, checkpoint gate positions, and reward definition define the task and must stay fixed.
- Students must not modify `rooms_layout.py`, `reward_config.py`, checkpoint coordinates, wall geometry, or reward terms for the submitted result.
- Students may modify the learning setup around the fixed task. Examples include PPO hyperparameters, training duration, entropy or exploration settings, network settings, action scaling, observation handling, or other variables that affect policy learning without changing the task itself.

- Any modification must be explained in the video report and compared against the assignment setup.
- The submitted code should be readable. Comments should explain non-obvious design decisions, not repeat what a nearby line of code already says.

Software Setup

The codebase was developed and tested on Ubuntu/Linux. Windows and macOS should also work through Conda in theory, but they were not tested end to end for this release. Platform setup references:

- Ubuntu/Linux Conda install guide: `https://docs.conda.io/projects/conda/en/latest/user-guide/install/linux.html`
- Windows Conda install guide: `https://docs.conda.io/projects/conda/en/latest/user-guide/install/windows.html`
- macOS Conda install guide: `https://docs.conda.io/projects/conda/en/latest/user-guide/install/macos.html`

The command blocks below were tested end to end on Ubuntu/Linux. On Windows, use the short setup note in `Exploring_agent_DRL/docs/windows_setup.md`. On macOS, use Terminal and follow the same `git clone`, `cd`, Conda, and Python commands after installing Conda; skip Linux package-manager commands such as `sudo apt`. Shell-specific commands such as `source <path-to-miniconda>/etc/profile.d/conda.sh` apply to Linux/macOS terminals only.

The assignment does not require heavy GPU compute. A good CPU is sufficient for training. An NVIDIA GPU may be used only if CUDA is installed correctly and PyTorch can detect it.

On Ubuntu/Linux, install Git if needed, then clone the assignment:

```
sudo apt update
sudo apt install git
git clone https://github.com/UAV-Centre-ITC/AI4R_explore_agent.git
cd AI4R_explore_agent
cd Exploring_agent_DRL
```

CPU environment:

```
conda env create -f environment.yml
conda activate aiar-rl-explore
```

NVIDIA GPU environment:

```
conda env create -f environment-gpu.yml
conda activate aiar-rl-explore-gpu
```

Check CUDA before using GPU training:

```
nvidia-smi
python -c "import torch; print(torch.cuda.is_available()); print(torch.cuda.
    get_device_name(0) if torch.cuda.is_available() else 'no CUDA')"
```

Use `-num-gpus 1` only when CUDA is available. Otherwise use `-num-gpus 0`.

Environment Interface

The environment follows the Gymnasium interaction pattern:

```
obs, info = env.reset()
obs, reward, terminated, truncated, info = env.step(action)
env.render()
env.close()
```

Training is headless by default, so no Pygame window is opened during training. Visual rendering is used for manual debugging and final rollout videos.

Action Space

The policy outputs two continuous actions:

$$a = [a_1, a_2]$$

- $a_1 > 0$: accelerate forward.
- $a_1 < 0$: reverse.
- $a_2 > 0$: rotate right.
- $a_2 < 0$: rotate left.

Both action values are clipped to the range $[-1, 1]$. The robot has damped motion, limited speed, steering limits, and a wall-clearance collision radius so that the visible robot body does not overlap walls.

Observation Space

The first ten observation values describe the robot's local motion and target direction:

$$O = [l_1, l_2, l_3, l_4, l_5, l_6, l_7, v, \alpha, \beta]$$

The current `2d_checkpoint_exploration` task uses an observation vector of shape $(27,)$. It contains:

- seven normalized range-ray distances $l_1, \ldots, l_7$;
- normalized robot speed v;
- α, the normalized angle between robot heading and velocity direction;
- β, the normalized angle between robot heading and the selected checkpoint target direction;
- checkpoint target slots, with the closest visible unvisited checkpoint used as the active target;
- overall checkpoint progress;
- stall time since the last new checkpoint.

The active checkpoint slot is represented as [relative angle, distance, visible flag]. The visible flag is positive only when the selected checkpoint can be seen without a wall blocking the line of sight. If the robot is too close to a wall, rays are blocked and checkpoints behind the wall are not visible. Unused slots are marked as not visible. This keeps the guidance simple when more than one checkpoint is visible.

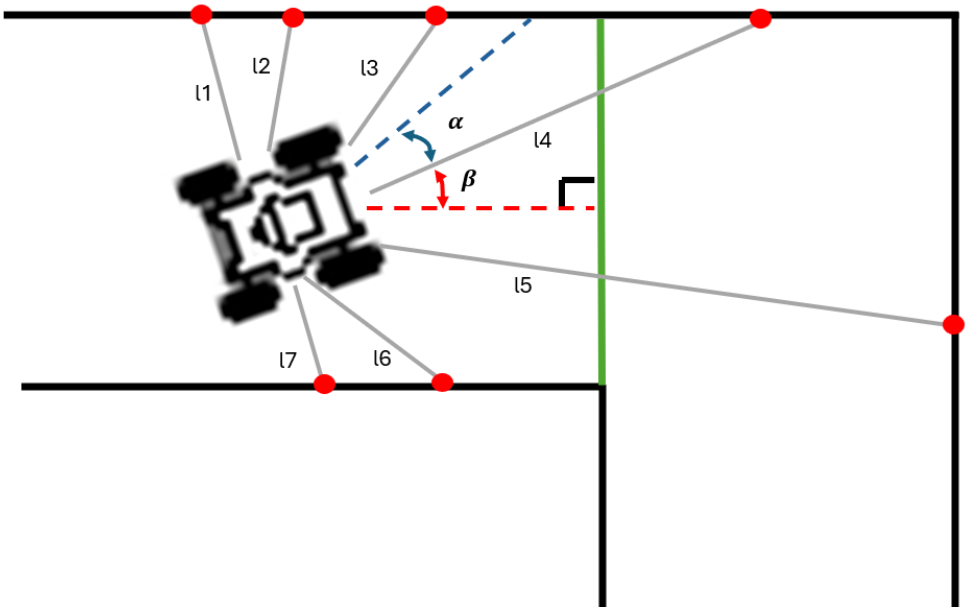

Figure 2: Observation-space illustration for the local ray, speed, heading, and target-direction terms.

Reward Function

The default reward mode is `coverage`. Reward constants are defined in:

```
Exploring_agent_DRL/Explore-Agent/explore_agent/envs/reward_config.py
```

The current reward definition is:

Term	Meaning
$+0.5$	crossing a new checkpoint for the first time
0.0	revisiting an already visited checkpoint
$+0.000$ to $+0.004$	getting closer to the selected visible checkpoint
-0.004	hovering in already explored space
-0.020	being blocked by a wall
-0.002	not getting closer to a visible unvisited checkpoint
bounded penalty	wall-contact penalty accumulating up to -0.5 between new checkpoints

Checkpoint reward is only given when the robot crosses a gate. Driving close to a checkpoint on the same side does not count. Once all checkpoints are crossed, the episode terminates with `done_reason="all_checkpoints"`. The small progress reward helps the agent learn the simple behavior of steering toward the selected red checkpoint before it has learned to cross gates reliably. The bounded wall-contact penalty resets after the robot reaches a new checkpoint. The extra -0.020 blocked-wall penalty discourages policies from pushing into walls after the bounded penalty has saturated.

Code Structure

Start by reading the following files:

- `Explore-Agent/explore_agent/envs/reward_config.py`: reward and task constants.

- `Explore-Agent/explore_agent/envs/rooms_layout.py`: fixed walls and checkpoints.
- `Explore-Agent/explore_agent/envs/exploring_gym.py`: environment dynamics, observations, collisions, rendering, and Gymnasium API.
- `Explore_PPO_agent_training.py`: PPO configuration and training loop.
- `Explore_DDPG_agent_training.py`: optional DDPG training script.
- `explore_agent_rollout.py`: visual and terminal evaluation of trained checkpoints.
- `run_assignment.py`: simple wrapper for checking, training, and rollout.

Quick Checks

Run a headless environment check:

```
python run_assignment.py check --steps 1000
```

Run manual driving:

```
python Explore-Agent/explore_agent/envs/start_human.py
```

Use the arrow keys to drive. Press `r` to reset.

Training PPO

Default PPO command. Start here with `entropy_coeff=0.1`:

```
python run_assignment.py train --iterations 500 --train-batch-size 2000 --sgd-
    minibatch-size 256 --num-sgd-iter 10 --num-workers 4 --num-gpus 0 --entropy-coeff
    0.1 --checkpoint-dir tmp/ppo_entropy_010
```

Training is much faster with parallel rollout workers. Check the number of available CPU processors:

```
python -c "import os; print(os.cpu_count())"
```

Use at least 4 workers when possible. On a stronger machine, start with at least half of the available processors, leaving the rest for the learner, operating system, and other applications.

During training, the log column `target>=6` shows whether the latest training batch contains an episode above the submission threshold. The `max%` column shows that batch's best episode reward as a percentage of the 10.0 maximum checkpoint reward.

Training Parameters

Parameter	Meaning
`-iterations`	Number of PPO collect-and-learn cycles. More iterations usually gives the policy more chances to improve.
`-train-batch-size`	Number of environment steps collected before one PPO update. The default 2000 is about five full 400-step training episodes.
`-sgd-minibatch-size`	Number of samples used in each optimizer minibatch after the full train batch is collected.
`-num-sgd-iter`	Number of optimization passes over the collected train batch.
`-entropy-coeff`	Entropy coefficient used by PPO. The provided setup uses 0.1; students should research and compare additional values before choosing the best final setting.
`-max-steps`	Maximum training episode length. Training uses 400 by default; final rollout may run up to 1000 steps.
`-spawn-mode`	Start-pose selection. Training uses `random` by default, sampling from fixed safe start poses. Rollout uses `fixed` by default for repeatable videos.
`-spawn-index`	Fixed start-pose index used when `-spawn-mode fixed`.
`-num-workers`	Number of Ray rollout workers. Use at least 4 when possible. For faster training, check the available processor count and start with at least half of that number.
`-num-gpus`	Use 1 only when CUDA is available. Otherwise use 0.

With the default training length:

$$500 \text{ iterations} \times 2000 \text{ environment steps per iteration} = 1{,}000{,}000 \text{ sampled steps}$$

Optional RL Algorithms

PPO is the recommended route and the provided report commands are written for PPO. Students may use another RL algorithm if they keep `rooms_layout.py`, `reward_config.py`, checkpoint gates, walls, and reward terms unchanged. If another algorithm is used for the submitted result, the report must explain the selected algorithm and repeat the exploration-parameter comparison using that algorithm's closest equivalent to `entropy_coeff`.

Algorithm	Exploration parameter to compare
PPO	`entropy_coeff`.
A3C	`entropy_coeff` in RLlib, although A3C is deprecated in the installed RLlib version.
SAC	Entropy temperature settings, such as `target_entropy` and `initial_alpha`, not PPO-style `entropy_coeff`.
DDPG	No entropy coefficient; compare action-noise settings such as `-exploration-initial-scale`, `-exploration-final-scale`, and `-exploration-scale-timesteps`.
TD3	No entropy coefficient; compare action-noise settings if a TD3 trainer is added.

Optional DDPG command:

```
python run_assignment.py train-ddpg --iterations 1000 --num-workers 4 --num-gpus 0 --
    checkpoint-dir tmp/ddpg_2d_checkpoint_exploration
```

Students may adjust these training values to reach the task goal, as long as the map geometry remains fixed and the changes are explained.

Continue Training from a Checkpoint

To continue training from an existing RLlib checkpoint, use `-resume-from` and save new checkpoints in a new directory:

```
python run_assignment.py train --iterations 300 --resume-from tmp/ppo_entropy_010/
    checkpoint_best --checkpoint-dir tmp/ppo_entropy_010_resume --num-workers 4 --num-
    gpus 0 --entropy-coeff 0.1 --train-batch-size 2000
```

Use the same fixed map when resuming. If the map, observation format, or reward mode is changed, old checkpoints may not be compatible or may behave differently from the submitted setup.

Testing a Trained Agent

Visual rollout:

```
python run_assignment.py rollout --checkpoint tmp/ppo_entropy_010/checkpoint_best --
    steps 1000 --max-steps 1000 --render-fps 30
```

To inspect a trained policy from the same random start distribution used during training:

```
python run_assignment.py rollout --checkpoint tmp/ppo_entropy_010/checkpoint_best --
    steps 1000 --max-steps 1000 --render-fps 30 --spawn-mode random
```

Terminal-only rollout:

```
python run_assignment.py rollout --checkpoint tmp/ppo_entropy_010/checkpoint_best --
    steps 1000 --max-steps 1000 --no-gui
```

The rollout prints cumulative reward, checkpoint coverage, maximum checkpoint reward, progress reward, hover penalty, collision penalty, and progress penalty. For assessment, use the best score reached at any time within the 1000-step rollout. It does not have to remain above 6.0 until the final frame. If the robot reaches a good score and then loses a lot of reward, try to troubleshoot and improve the behavior, then explain the failure after the rollout video.

Required Experiments

Students must complete at least the following:

1. Train PPO using the provided command as a baseline.
2. Research how `entropy_coeff` and other PPO hyperparameters affect exploration and learning.
3. Tune the training setup to reach more than 6.0 during a 1000-step rollout.
4. Train one comparison run with `entropy_coeff=0.0`.
5. Compare reward curves, checkpoint coverage, wall-contact behavior, and visual rollout behavior.

6. Explain the behavior difference between the best setup and the `entropy_coeff=0.0` version.

PPO is the default supported algorithm for the assignment. Students may try another RL algorithm if they keep the map, checkpoints, and reward definition fixed. If another algorithm is used for the submitted result, the required comparison should use the closest exploration parameter for that algorithm.

First report what you learned about entropy regularization in PPO and how it relates to exploration. Then explain which entropy value and other hyperparameters you selected for the best setup, and why. Compare that best setup against one `entropy_coeff=0.0` run and explain the behavior difference. List any hyperparameters changed from the provided command, such as `-iterations`, `-train-batch-size`, `-sgd-minibatch-size`, `-num-sgd-iter`, network settings, or other PPO/training options. For each change, explain why it was made and how it affected the score or rollout behavior. If another algorithm is used, report the equivalent algorithm-specific hyperparameters instead.

For PPO, run this comparison case against the best setup:

```
python run_assignment.py train --iterations 500 --train-batch-size 2000 --sgd-
    minibatch-size 256 --num-sgd-iter 10 --num-workers 4 --num-gpus 0 --entropy-coeff
    0.0 --checkpoint-dir tmp/ppo_entropy_000
```

Test the best checkpoint and the `entropy_coeff=0.0` checkpoint, then report what changed in the reward curve and rollout behavior. If a different RL algorithm is used, run the same kind of comparison with its closest low/no-exploration parameter and explain which parameter was changed.

Deliverables

Students must submit:

- a short video report, 5 minutes or less, showing the best rollout with the explanations listed below;
- the best score reached at any time during the 1000-step rollout;
- the final score, if it differs from the best score;
- a comparison between the best setup and the `entropy_coeff=0.0` comparison run, discussed or presented in the video report;
- a short explanation of the entropy value and other hyperparameters changed to reach the best score, and why those changes were made;
- a short explanation of successful behavior and failure cases;
- a zip file containing the source code used for the run and the trained checkpoint used to record the submitted video.

The video only needs to show the best rollout. Discuss or present the `entropy_coeff=0.0` comparison using concise plots, tables, or spoken explanation; there is no need to include the full comparison rollout. If a short clip from the comparison rollout supports a behavior pattern discussed in the report, it can be included briefly.

Suggested video structure:

1. show the best rollout first;
2. report the best score, final score, and checkpoint count;

3. explain the main behavior observed in the rollout;
4. explain any failure near the end, such as wall contact, stalling, or a large reward drop;
5. summarize the most important hyperparameter changes and entropy comparison.

Discussion Questions

The video report should answer the following:

- What was the best reward reached during the 1000-step rollout?
- What was the final reward, and did it differ from the best reward?
- How many checkpoints were crossed?
- Did the robot enter multiple rooms or stay in one local area?
- Did it slow down enough to cross side checkpoints?
- Did it get stuck, oscillate, or collide with walls?
- Which entropy value and other hyperparameters gave the best run, and why?
- How did changing `entropy_coeff` affect exploration and stability?
- Which PPO or training changes improved the behavior most clearly?

Assessment Summary

- Maximum checkpoint reward: 10.0.
- Minimum target for submission: more than 6.0 at any point during the 1000-step rollout.
- The score does not need to stay above 6.0 until the end, but large drops must be explained.
- The fixed map layout must not be changed.
- The submitted checkpoint must match the video shown.
- The explanation is part of the assessment; do not submit only a reward number.